

Formal Languages and Compilers
Prof. Breveglieri, Morzenti, Agosta
Written exam: laboratory question

20/01/2023

Time: 60 minutes. Textbooks and notes can be used. Pencil writing is allowed.

Important: Write your name on any additional sheet.

SURNAME (Cognome):

NAME (Nome):

Matricola:or Person Code:

Instructor: ☐ Prof. Breviglieri ☐ Prof. Morzenti ☐ Prof. Agosta

The laboratory question must be answered taking into account the implementation of the Acse compiler given with the exam text.

Modify the specification of the lexical analyser (*flex* input) and the syntactic analyser (*bison* input) and any other source file required to extend the Lance language with the ability to check if an array element exists or not before performing an assignment. This operation is performed by an expression operator called **inbounds**. Its syntax consists of the **inbounds** keyword itself, followed by two parameters enclosed in parenthesis: a *destination* and a *source*, in this order. The parameters are separated by "=", mimicking the syntax used for assignments. If the left-hand side of "=" is not a scalar variable, or if the right-hand side of "=" is not an array element, then the compilation fails.

When **inbounds** appears in an expression, it verifies that the index of the specified array element is within bounds, i.e. if the index is greater or equal than zero and lesser than the size of the array. If this condition is satisfied, the operator copies the array element to the scalar and returns 1, otherwise it returns zero and no copy is performed.

The following code snippet exemplifies the operation and syntax of **inbounds**. The first *if* statement checks the validity of index 0 (zero) in array *a*. The index is within bounds, therefore the value of *a*[0], which is 8, is put into *v* and the first section of the *if* is executed. In the following statement, **inbounds** appears in the expression argument of a **write** and checks if the index $-1 + v$ is valid. The variable *v* has just been assigned to 8, so the index is 7. Since this index is too large, a zero is printed and *v* is unmodified. Finally, the program uses **inbounds** to print the last 3 values in *a*. At every iteration the index *i* is checked, and as soon as it is not in bounds the condition of the *while* statement becomes false and the loop terminates.

```
int v, i, a[5];
a[0]=8; a[1]=6; a[2]=9; a[3]=4; a[4]=7;

if (inbounds(v = a[0])) {
    write(1);           // v = a[0], writes 1
} else {
    write(2);
}

write(inbounds(v = a[-1+v])); // check fails, writes 0

i = 2;
while (inbounds(v = a[i])) {
    write(v);
    i = i + 1;
}                               // the loop writes 9, 4, 7
```

1. Define the tokens (and the related declarations in **Acse.lex** and **Acse.y**). (2 points)
2. Define the syntactic rules or the modifications required to the existing ones. (3 points)
3. Define the semantic actions needed to implement the required functionality. (20 points)

Solution

The solution is shown below in *diff* format. All lines that begin with '+' were added, while the lines that begin with '-' were removed. **It is not required and it is not encouraged to provide a solution in *diff* format to get the maximum grade.**

```
diff --git a/acse/Acse.lex b/acse/Acse.lex
index 35f9b73..6212219 100644
--- a/acse/Acse.lex
+++ b/acse/Acse.lex
@@ -93,6 +93,7 @@ ID      [a-zA-Z_][a-zA-Z0-9_]*
"return"      { return RETURN; }
"read"        { return READ; }
"write"       { return WRITE; }
+"inbounds"   { return INBOUNDS; }

{ID}          { yylval.svalue=strdup(yytext); return IDENTIFIER; }
{DIGIT}+      { yylval.intval = atoi( yytext ); }
diff --git a/acse/Acse.y b/acse/Acse.y
index 0636dc6..4dcefe2 100644
--- a/acse/Acse.y
+++ b/acse/Acse.y
@@ -125,6 +125,7 @@ extern void yyerror(const char*);
%token RETURN
%token READ
%token WRITE
+%token INBOUNDS

%token <label> DO
%token <while_stmt> WHILE
@@ -458,7 +459,72 @@ write_statement : WRITE LPAR exp RPAR
                }
;

-exp: NUMBER { $$ = create_expression($1, IMMEDIATE); }
+exp: INBOUNDS LPAR IDENTIFIER ASSIGN IDENTIFIER LSQUARE exp RSQUARE RPAR
+  {
+    /* Check if the variable in the left-hand side of the assignment is in
+     * fact a scalar variable */
+    t_axe_variable *v_dst = getVariable(program, $3);
+    if (!v_dst || v_dst->isArray) {
+      yyerror("destination variable is not a scalar");
+      YYERROR;
+    }
+    /* Check if the variable in the right-hand side of the assignment is in
+     * fact an array-typed variable */
+    t_axe_variable *v_src = getVariable(program, $5);
+    if (!v_src || !v_src->isArray) {
+      yyerror("source variable is not an array");
+      YYERROR;
+    }
+    /* Generate code to compute the validity condition of the array index
+     *   index >= 0 && index < arraySize
+     *   using handle_binary_comparison and handle_bin_numeric_op.
+     *   This will also automatically take care of constant folding here. */
+    t_axe_expression geZero = handle_binary_comparison(program,
+      $7, create_expression(0, IMMEDIATE), _GTEQ_);
+    t_axe_expression ltArraySize = handle_binary_comparison(program,
+      $7, create_expression(v_src->arraySize, IMMEDIATE), _LT_);
+    t_axe_expression indexValid = handle_bin_numeric_op(program,
```

```

+         geZero, ltArraySize, ANDL);
+
+     /* Get the register associated with the variable in the left-hand side
+      * of the assignment. */
+     int r_dest = get_symbol_location(program, $3, 0);
+
+     /* Check if the validity condition was constant-folded or not. */
+     if (indexValid.expression_type == IMMEDIATE) {
+         /* The condition was constant-folded, and therefore its value is
+          * constant. There is no need to generate any branch, but we must
+          * check the validity of the index at compile time. */
+         if (indexValid.value != 0) {
+             /* The index is valid, generate code to load the indexed array
+              * element and store it into the specified variable. */
+             int r_val = loadArrayElement(program, $5, $7);
+             gen_addi_instruction(program, r_dest, r_val, 0);
+         }
+     } else {
+         /* The condition is in a register. Generate code to check the index
+          * validity condition and skip the assignment if the condition is
+          * false. */
+         gen_subi_instruction(program, REG_0, indexValid.value, 0);
+         t_axe_label *l_skip = newLabel(program);
+         gen_beq_instruction(program, l_skip, 0);
+         /* Generate code which will load the indexed array element and store
+          * it into the specified variable if the index is within bounds. */
+         int r_val = loadArrayElement(program, $5, $7);
+         gen_addi_instruction(program, r_dest, r_val, 0);
+         /* Generate (assign) the label where the branch generated above will
+          * jump when the condition is false, skipping the assignment. */
+         assignLabel(program, l_skip);
+     }
+
+     /* The value of the expression is just the validity of the index. */
+     $$ = indexValid;
+     /* Free the dynamically-allocated identifiers. */
+     free($3);
+     free($5);
+ }
+ | NUMBER      { $$ = create_expression ($1, IMMEDIATE); }
+ | IDENTIFIER  {
+
+         int location;

```

```

diff --git a/tests/inbounds/inbounds.src b/tests/inbounds/inbounds.src
new file mode 100644
index 0000000..641279d
--- /dev/null
+++ b/tests/inbounds/inbounds.src
@@ -0,0 +1,20 @@
+int v, i, a[5];
+ a[0]=8;
+ a[1]=6;
+ a[2]=9;
+ a[3]=4;
+ a[4]=7;
+
+ if (inbounds(v = a[0])) {
+     write(1);           // v = a[0], writes 1
+ } else {
+     write(2);
+ }
+
+ write(inbounds(v = a[-1])); // check fails, writes 0
+
+ i = 2;
+ while (inbounds(v = a[i])) {
+     write(v);
+     i = i + 1;
+ }
+
+ // the loop writes 9, 4, 7

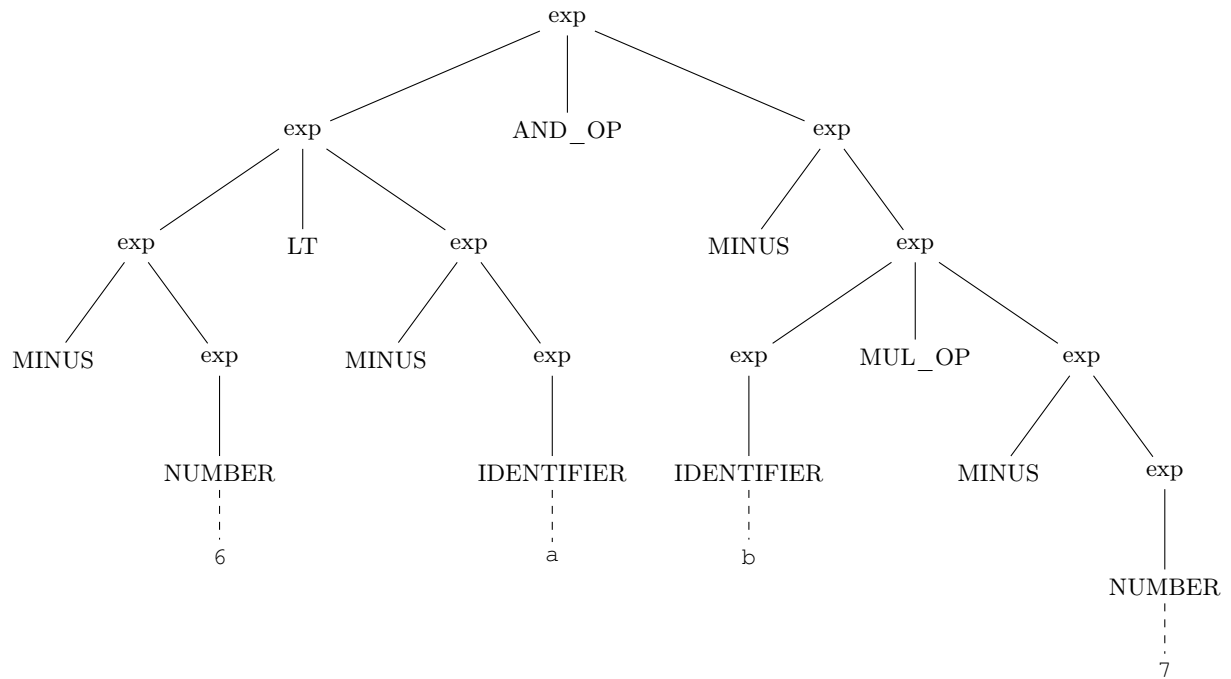
```

4. Given the following Lance code snippet:

-6 < -a & -b * -7

write down the syntactic tree generated during the parsing with the Bison grammar described in Acse.y *starting from the exp nonterminal*. (5 points)

Solution



5. (**Bonus**) Describe the modifications to the specification and implementation of the `inbounds` operator required for supporting assignments to a second array.